Loading Data from `Rest API` to `Supabase` in Python with `dlt`

ABOUT OUR REST_API VERIFIED SOURCE

This example demonstrates how to use the rest_api to retrieve data from the GitHub Rest API, but will work with any HTTP Rest API. Please read:

- The rest_api docs to learn how to configure this verified source
- The OpenAPI generator docs to learn how to automatically configure a dlt rest_api source from an OpenAPI spec
- Our cool google colab example demonstrating the generator and the rest_api source

TIP

We will be using the dlt PostgreSQL destination to connect to Supabase. You can get the connection string for your Supabase database as described in the Supabase Docs.

NEED HELP DEPLOYING THESE PIPELINES, OR FIGURING OUT HOW TO RUN THEM IN YOUR DATA STACK?

Join our Slack community or book a call with our support engineer Violetta.

The `rest_api` verified source supports getting data from any HTTP `Rest API` to `Supabase`. `Supabase` is an open-source Firebase alternative that offers a Postgres database, Authentication, instant APIs, Edge Functions, Realtime subscriptions, Storage, and Vector embeddings. Using the open-source Python library `dlt`, you can easily integrate and manage your data pipeline. For further information on the `rest_api` source, visit this link.

`dlt` Key Features

- **Automated Maintenance**: With schema inference, evolution, and alerts, `dlt` simplifies maintenance. Learn more
- **Run Anywhere Python Runs**: Deploy on Airflow, serverless functions, notebooks, and more. Find out how
- **User-Friendly Interface**: Declarative interface that lowers knowledge barriers for beginners and empowers professionals. Explore the interface
- **Securely Handle Secrets**: Securely store sensitive information like access tokens in the `.dlt` folder. Handle secrets securely
- **Incremental Loading**: Load only new data and deduplicate existing data with incremental loading. Understand incremental loading

Getting started with your pipeline locally

0. Prerequisites

`dlt` requires Python 3.8 or higher. Additionally, you need to have the `pip` package manager installed, and we recommend using a virtual environment to manage your dependencies. You can learn more about preparing your computer for dlt in our installation reference.

1. Install dlt

First you need to install the `dlt` library with the correct extras for `Supabase`:

```
pip install "dlt[postgres]"
```

The `dlt` cli has a useful command to get you started with any combination of source and destination. For this example, we want to load data from `Rest API` to `Supabase`. You can run the following commands to create a starting point for loading data from `Rest API` to `Supabase`:

```
# create a new directory
mkdir rest_api_pipeline
cd rest_api_pipeline
# initialize a new pipeline with your source and destination
dlt init rest_api postgres
```

```
# install the required dependencies
pip install -r requirements.txt
```

The last command will install the required dependencies for your pipeline. The dependencies are listed in the `requirements.txt`:

```
dlt[postgres]>=0.4.11
```

You now have the following folder structure in your project:

```
rest_api_pipeline/
├── .dlt/
│   ├── config.toml          # configs for your pipeline
│   └── secrets.toml         # secrets for your pipeline
├── rest_api/                   # folder with source specific files
│   └── ...
├── rest_api_pipeline.py        # your main pipeline script
├── requirements.txt         # dependencies for your pipeline
└── .gitignore               # ignore files for git (not required)
```

2. Configuring your source and destination credentials

The `dlt` cli will have created a `.dlt` directory in your project folder. This directory contains a `config.toml` file and a `secrets.toml` file that you can use to configure your pipeline. The automatically created version of these files look like this:

generated config.toml

```
# put your configuration values here

[runtime]
log_level="WARNING"  # the system log level of dlt
# use the dlthub_telemetry setting to enable/disable anonymous usage data
reporting, see https://dlthub.com/docs/telemetry
dlthub_telemetry = true
```

generated secrets.toml

```
# put your secret values and credentials here. do not share this file and do
not push it to github

[sources.rest_api]
github_token = "github_token" # please set me up!

[destination.postgres]
dataset_name = "dataset_name" # please set me up!

[destination.postgres.credentials]
database = "database" # please set me up!
password = "password" # please set me up!
username = "username" # please set me up!
host = "host" # please set me up!
port = 5432
connect_timeout = 15
```

2.1. Adjust the generated code to your usecase

> **FURTHER HELP SETTING UP YOUR SOURCE AND DESTINATIONS**
>
> - Read more about setting up the `Rest API` source in our docs.
> - Read more about setting up the `Supabase` destination in our docs.

3. Running your pipeline for the first time

The `dlt` cli has also created a main pipeline script for you at `rest_api_pipeline.py`, as well as a folder `rest_api` that contains additional python files for your source. These files are your local copies which you can modify to fit your needs. In some cases you may find that you only need to do small changes to your pipelines or add some configurations, in other cases these files can serve as a working starting point for your code, but will need to be adjusted to do what you need them to do.

The main pipeline script will look something like this:

```
from typing import Any

import dlt
```

```python
from rest_api import (
    RESTAPIConfig,
    check_connection,
    rest_api_source,
    rest_api_resources,
)


@dlt.source
def github_source(github_token: str = dlt.secrets.value) -> Any:
    # Create a REST API configuration for the GitHub API
    # Use RESTAPIConfig to get autocompletion and type checking
    config: RESTAPIConfig = {
        "client": {
            "base_url": "https://api.github.com/repos/dlt-hub/dlt/",
            "auth": {
                "type": "bearer",
                "token": github_token,
            },
        },
        # The default configuration for all resources and their endpoints
        "resource_defaults": {
            "primary_key": "id",
            "write_disposition": "merge",
            "endpoint": {
                "params": {
                    "per_page": 100,
                },
            },
        },
        "resources": [
            # This is a simple resource definition,
            # that uses the endpoint path as a resource name:
            # "pulls",
            # Alternatively, you can define the endpoint as a dictionary
            # {
            #     "name": "pulls", # <- Name of the resource
            #     "endpoint": "pulls",  # <- This is the endpoint path
            # }
            # Or use a more detailed configuration:
            {
                "name": "issues",
                "endpoint": {
                    "path": "issues",
                    # Query parameters for the endpoint
                    "params": {
                        "sort": "updated",
```

```
                        "direction": "desc",
                        "state": "open",
                        # Define `since` as a special parameter
                        # to incrementally load data from the API.
                        # This works by getting the updated_at value
                        # from the previous response data and using this value
                        # for the `since` query parameter in the next request.
                        "since": {
                            "type": "incremental",
                            "cursor_path": "updated_at",
                            "initial_value": "2024-01-25T11:21:28Z",
                        },
                    },
                },
            },
            # The following is an example of a resource that uses
            # a parent resource (`issues`) to get the `issue_number`
            # and include it in the endpoint path:
            {
                "name": "issue_comments",
                "endpoint": {
                    # The placeholder {issue_number} will be resolved
                    # from the parent resource
                    "path": "issues/{issue_number}/comments",
                    "params": {
                        # The value of `issue_number` will be taken
                        # from the `number` field in the `issues` resource
                        "issue_number": {
                            "type": "resolve",
                            "resource": "issues",
                            "field": "number",
                        }
                    },
                },
                # Include data from `id` field of the parent resource
                # in the child data. The field name in the child data
                # will be called `_issues_id` (_{resource_name}_{field_name})
                "include_from_parent": ["id"],
            },
        ],
    }

    yield from rest_api_resources(config)


def load_github() -> None:
    pipeline = dlt.pipeline(
```

```
        pipeline_name="rest_api_github",
        destination='postgres',
        dataset_name="rest_api_data",
    )

    load_info = pipeline.run(github_source())
    print(load_info)


def load_pokemon() -> None:
    pipeline = dlt.pipeline(
        pipeline_name="rest_api_pokemon",
        destination='postgres',
        dataset_name="rest_api_data",
    )

    pokemon_source = rest_api_source(
        {
            "client": {
                "base_url": "https://pokeapi.co/api/v2/",
                # If you leave out the paginator, it will be inferred from the
API:
                # paginator: "json_response",
            },
            "resource_defaults": {
                "endpoint": {
                    "params": {
                        "limit": 1000,
                    },
                },
            },
            "resources": [
                "pokemon",
                "berry",
                "location",
            ],
        }
    )

    def check_network_and_authentication() -> None:
        (can_connect, error_msg) = check_connection(
            pokemon_source,
            "not_existing_endpoint",
        )
        if not can_connect:
            pass  # do something with the error message
```

```
    check_network_and_authentication()

    load_info = pipeline.run(pokemon_source)
    print(load_info)


if __name__ == "__main__":
    load_github()
    load_pokemon()
```

Provided you have set up your credentials, you can run your pipeline like a regular python script with the following command:

```
python rest_api_pipeline.py
```

4. Inspecting your load result

You can now inspect the state of your pipeline with the `dlt` cli:

```
dlt pipeline rest_api_github info
```

You can also use streamlit to inspect the contents of your `Supabase` destination for this:

```
# install streamlit
pip install streamlit
# run the streamlit app for your pipeline with the dlt cli:
dlt pipeline rest_api_github show
```

5. Next steps to get your pipeline running in production

One of the beauties of `dlt` is, that we are just a plain Python library, so you can run your pipeline in any environment that supports Python >= 3.8. We have a couple of helpers and guides in our docs to get you there:

The Deploy section will show you how to deploy your pipeline to

- **Deploy with GitHub Actions**: Learn how to set up and deploy your `dlt` pipeline using GitHub Actions for automated workflows. Read more
- **Deploy with Airflow and Google Composer**: Follow this guide to deploy your `dlt` pipeline using Airflow and Google Composer for managed workflow orchestration. Read more
- **Deploy with Google Cloud Functions**: Discover how to deploy your `dlt` pipeline using Google Cloud Functions for serverless execution. Read more
- **Explore more deployment options**: Check out other methods to deploy your `dlt` pipeline, including various cloud services and orchestration tools. Read more

The running in production section will teach you about:

- **How to Monitor your pipeline**: Learn how to effectively monitor your `dlt` pipeline in production to ensure smooth and reliable operation. How to Monitor your pipeline
- **Set up alerts**: Set up alerts to get notified of any issues or important events occurring in your `dlt` pipeline. Set up alerts
- **And set up tracing**: Implement tracing to gain insights into the performance and behavior of your `dlt` pipeline. And set up tracing

Available Sources and Resources

For this verified source the following sources and resources are available

Source `github_source`

"Rest API Source for GitHub, providing detailed data on issues and related comments."

Resource Name	Write Disposition	Description
`issue_comments`	`merge`	Contains information about the issue comments including the author, body of the comment, created date, and user details among other data.

Resource Name	Write Disposition	Description
`issues`	`merge`	Contains information about the issues including the assignee details, author, body of the issue, comments, created date, and user details among other data.

Additional pipeline guides

- Load data from Looker to AlloyDB in python with dlt
- Load data from Salesforce to EDB BigAnimal in python with dlt
- Load data from Aladtec to Snowflake in python with dlt
- Load data from Trello to CockroachDB in python with dlt
- Load data from Box Platform API to ClickHouse in python with dlt
- Load data from Salesforce to Timescale in python with dlt
- Load data from Salesforce to Microsoft SQL Server in python with dlt
- Load data from Adobe Commerce (Magento) to Dremio in python with dlt
- Load data from Jira to Microsoft SQL Server in python with dlt
- Load data from The Local Filesystem to AWS S3 in python with dlt

Edit this page

Last updated on **Aug 7, 2024** by **Zaeem Athar**